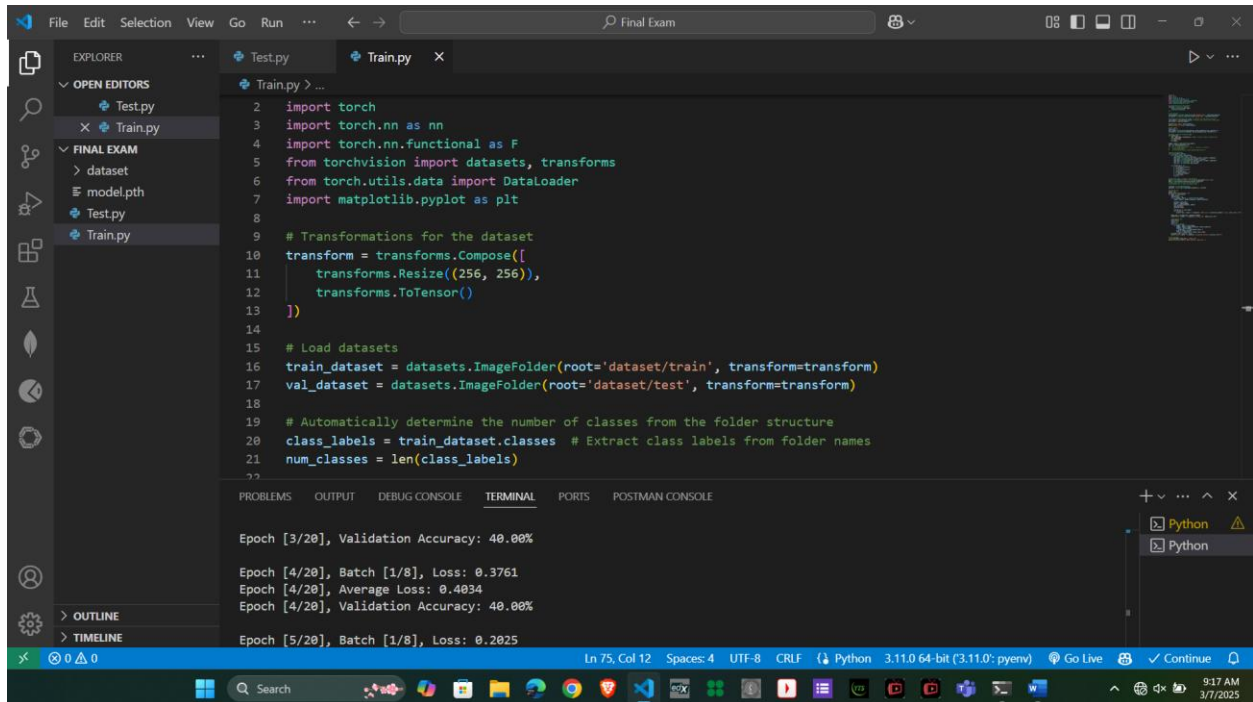


Part 2: AI Technique Problem coding

1. Baseline Training



```
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 from torchvision import datasets, transforms
6 from torch.utils.data import DataLoader
7 import matplotlib.pyplot as plt
8
9 # Transformations for the dataset
10 transform = transforms.Compose([
11     transforms.Resize((256, 256)),
12     transforms.ToTensor()
13 ])
14
15 # Load datasets
16 train_dataset = datasets.ImageFolder(root='dataset/train', transform=transform)
17 val_dataset = datasets.ImageFolder(root='dataset/test', transform=transform)
18
19 # Automatically determine the number of classes from the folder structure
20 class_labels = train_dataset.classes # Extract class labels from folder names
21 num_classes = len(class_labels)
22
```

Epoch [3/20], Validation Accuracy: 40.00%

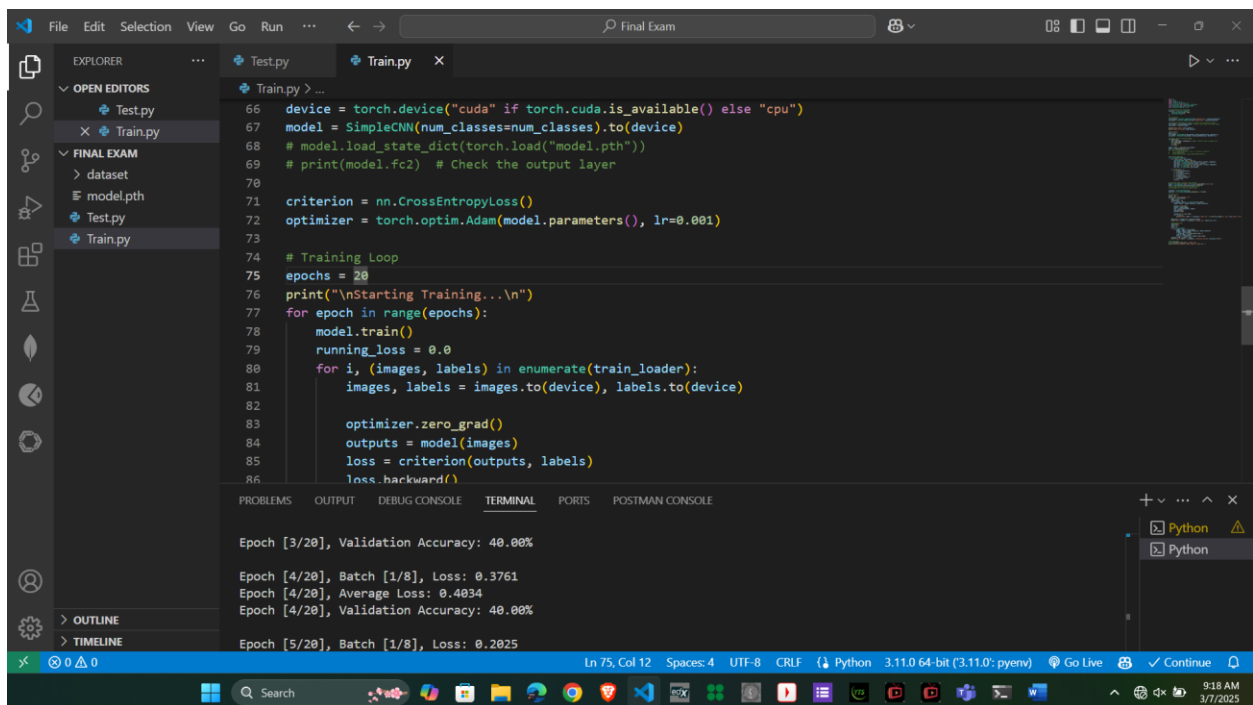
Epoch [4/20], Batch [1/8], Loss: 0.3761

Epoch [4/20], Average Loss: 0.4034

Epoch [4/20], Validation Accuracy: 40.00%

Epoch [5/20], Batch [1/8], Loss: 0.2025

Using Epochs= 20 and CrossEntropyLoss



```
66 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
67 model = SimpleCNN(num_classes=num_classes).to(device)
68 # model.load_state_dict(torch.load("model.pth"))
69 # print(model.fc2) # Check the output layer
70
71 criterion = nn.CrossEntropyLoss()
72 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
73
74 # Training Loop
75 epochs = 20
76 print("\nStarting Training...\n")
77 for epoch in range(epochs):
78     model.train()
79     running_loss = 0.0
80     for i, (images, labels) in enumerate(train_loader):
81         images, labels = images.to(device), labels.to(device)
82
83         optimizer.zero_grad()
84         outputs = model(images)
85         loss = criterion(outputs, labels)
86         loss.backward()
```

Epoch [3/20], Validation Accuracy: 40.00%

Epoch [4/20], Batch [1/8], Loss: 0.3761

Epoch [4/20], Average Loss: 0.4034

Epoch [4/20], Validation Accuracy: 40.00%

Epoch [5/20], Batch [1/8], Loss: 0.2025

Final Exam

EXPLORER

- OPEN EDITORS
 - Test.py
 - Arjan_Train.ipyn...
 - Train.py
- FINAL EXAM
 - dataset
 - model.pth
 - Test.py
 - Train.py
- OUTLINE
- TIMELINE

TERMINAL

```
.11.0/python.exe "c:/Users/Admin/OneDrive - Assumption University/Desktop/AI concept/Final Exam/Train.py"
Class Labels: ['Bridge', 'Crosswalk', 'Stair']
Number of Classes: 3
torch.Size([4, 3, 256, 256])

Starting Training...

Epoch [1/20], Batch [1/8], Loss: 1.0756
Epoch [1/20], Average Loss: 1.9707
Epoch [1/20], Validation Accuracy: 50.00%

Epoch [2/20], Batch [1/8], Loss: 0.6487
Epoch [2/20], Average Loss: 0.6711
Epoch [2/20], Validation Accuracy: 70.00%

Epoch [3/20], Batch [1/8], Loss: 0.2377
Epoch [3/20], Average Loss: 0.4248
Epoch [3/20], Validation Accuracy: 70.00%

Epoch [4/20], Batch [1/8], Loss: 0.2872
Epoch [4/20], Average Loss: 0.4094
Epoch [4/20], Validation Accuracy: 50.00%

Epoch [5/20], Batch [1/8], Loss: 0.2645
Epoch [5/20], Average Loss: 0.2598
Epoch [5/20], Validation Accuracy: 60.00%

Epoch [6/20], Batch [1/8], Loss: 0.0055
Epoch [6/20], Average Loss: 0.0381
Epoch [6/20], Validation Accuracy: 70.00%

Epoch [7/20], Batch [1/8], Loss: 0.0089
Epoch [7/20], Average Loss: 0.0445
```

Ln 46, Col 53 Spaces: 4 UTF-8 CRLF Python 3.11.0 64-bit (3.11.0: pyenv) Go Live Continue

Final Exam

EXPLORER

- OPEN EDITORS
 - Test.py
 - Arjan_Train.ipyn...
 - Train.py
- FINAL EXAM
 - dataset
 - model.pth
 - Test.py
 - Train.py
- OUTLINE
- TIMELINE

TERMINAL

```
Epoch [8/20], Batch [1/8], Loss: 0.0004
Epoch [8/20], Average Loss: 0.0199
Epoch [8/20], Validation Accuracy: 70.00%

Epoch [9/20], Batch [1/8], Loss: 0.0011
Epoch [9/20], Average Loss: 0.0694
Epoch [9/20], Validation Accuracy: 70.00%

Epoch [10/20], Batch [1/8], Loss: 0.0028
Epoch [10/20], Average Loss: 0.0025
Epoch [10/20], Validation Accuracy: 60.00%

Epoch [11/20], Batch [1/8], Loss: 0.0031
Epoch [11/20], Average Loss: 0.1239
Epoch [11/20], Validation Accuracy: 60.00%

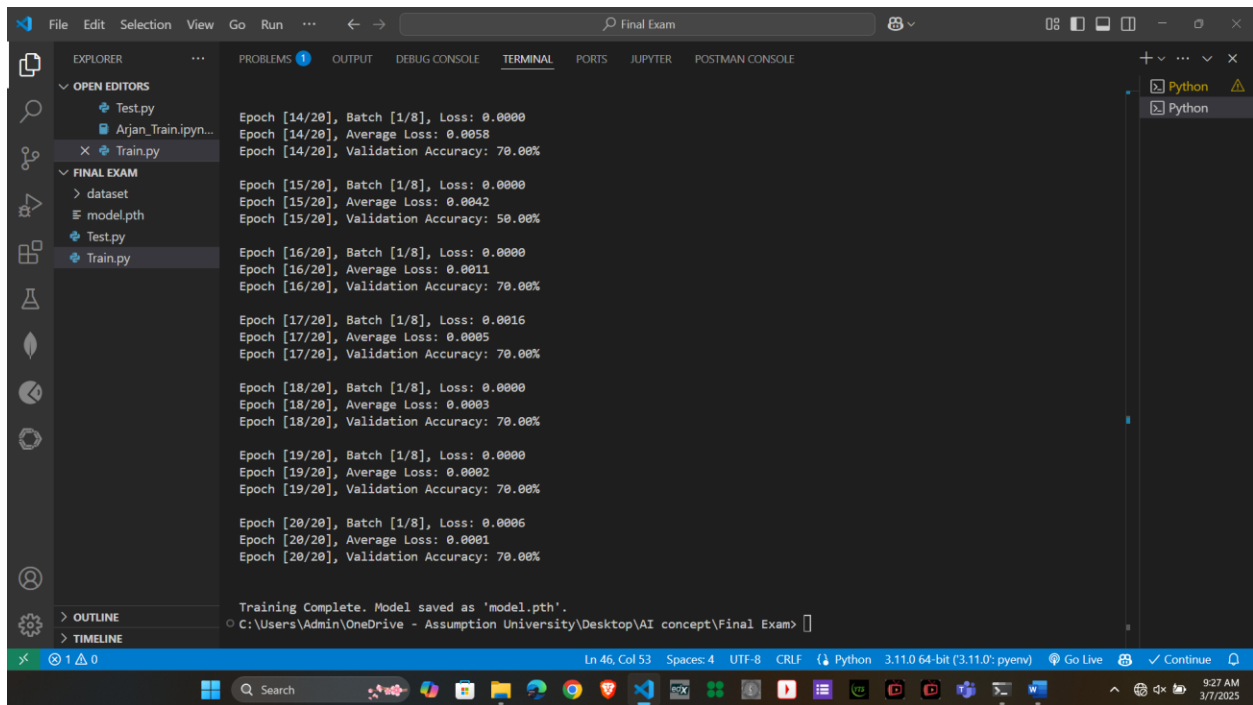
Epoch [12/20], Batch [1/8], Loss: 0.0006
Epoch [12/20], Average Loss: 0.0126
Epoch [12/20], Validation Accuracy: 60.00%

Epoch [13/20], Batch [1/8], Loss: 0.0008
Epoch [13/20], Average Loss: 0.0005
Epoch [13/20], Validation Accuracy: 60.00%

Epoch [14/20], Batch [1/8], Loss: 0.0000
Epoch [14/20], Average Loss: 0.0058
Epoch [14/20], Validation Accuracy: 70.00%

Epoch [15/20], Batch [1/8], Loss: 0.0000
Epoch [15/20], Average Loss: 0.0042
Epoch [15/20], Validation Accuracy: 50.00%
```

Ln 46, Col 53 Spaces: 4 UTF-8 CRLF Python 3.11.0 64-bit (3.11.0: pyenv) Go Live Continue



The screenshot shows the Visual Studio Code interface with the terminal window open. The terminal displays the output of a training script, showing progress for epochs 14 through 20. The output includes batch numbers, loss values, average loss, and validation accuracy. The training is complete, and the model is saved as 'model.pth'.

```
Epoch [14/20], Batch [1/8], Loss: 0.0000
Epoch [14/20], Average Loss: 0.0050
Epoch [14/20], Validation Accuracy: 70.00%

Epoch [15/20], Batch [1/8], Loss: 0.0000
Epoch [15/20], Average Loss: 0.0042
Epoch [15/20], Validation Accuracy: 50.00%

Epoch [16/20], Batch [1/8], Loss: 0.0000
Epoch [16/20], Average Loss: 0.0011
Epoch [16/20], Validation Accuracy: 70.00%

Epoch [17/20], Batch [1/8], Loss: 0.0016
Epoch [17/20], Average Loss: 0.0005
Epoch [17/20], Validation Accuracy: 70.00%

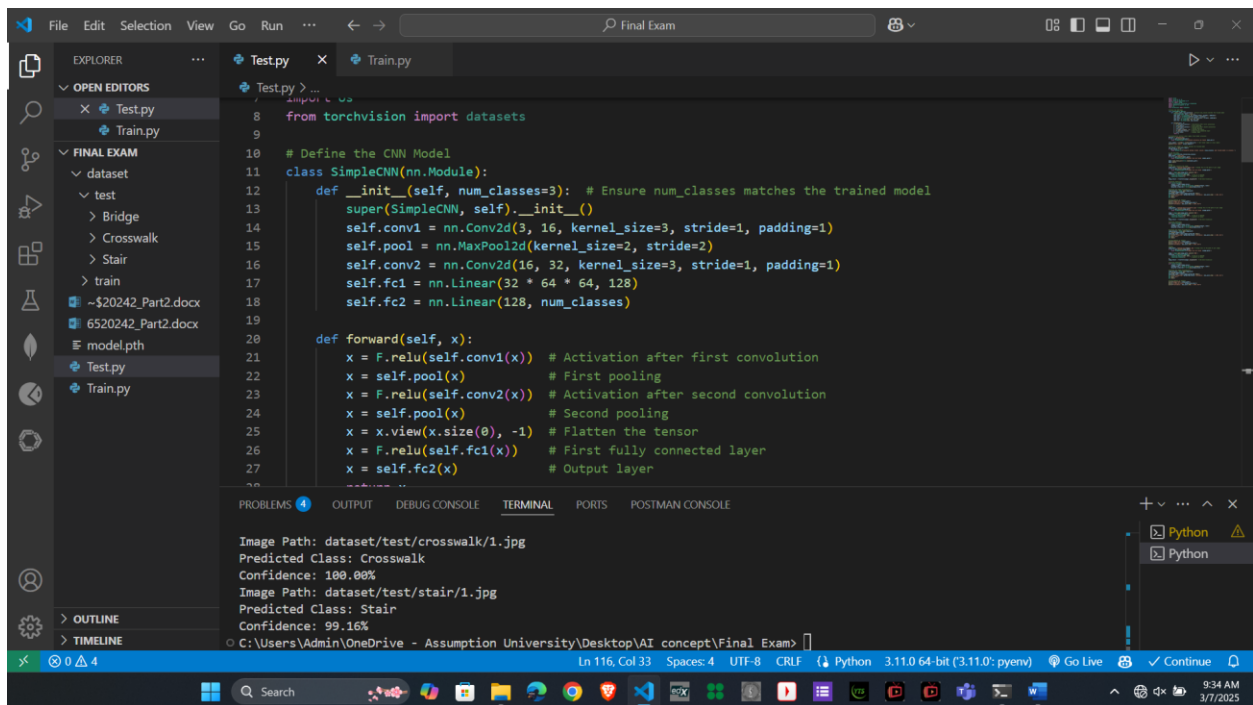
Epoch [18/20], Batch [1/8], Loss: 0.0000
Epoch [18/20], Average Loss: 0.0003
Epoch [18/20], Validation Accuracy: 70.00%

Epoch [19/20], Batch [1/8], Loss: 0.0000
Epoch [19/20], Average Loss: 0.0002
Epoch [19/20], Validation Accuracy: 70.00%

Epoch [20/20], Batch [1/8], Loss: 0.0006
Epoch [20/20], Average Loss: 0.0001
Epoch [20/20], Validation Accuracy: 70.00%

Training Complete. Model saved as 'model.pth'.
C:\Users\Admin\OneDrive - Assumption University\Desktop\AI concept\Final Exam>
```

2. Augmented Training



The screenshot shows the Visual Studio Code interface with the 'Test.py' file open. The code defines a SimpleCNN model and evaluates it on a test set. The output in the terminal shows the predicted class and confidence for two images: 'crosswalk/1.jpg' and 'stair/1.jpg'.

```
from torchvision import datasets

# Define the CNN Model
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=3): # Ensure num_classes matches the trained model
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.fc1 = nn.Linear(32 * 64 * 64, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = F.relu(self.conv1(x)) # Activation after first convolution
        x = self.pool(x) # First pooling
        x = F.relu(self.conv2(x)) # Activation after second convolution
        x = self.pool(x) # Second pooling
        x = x.view(x.size(0), -1) # Flatten the tensor
        x = F.relu(self.fc1(x)) # First fully connected layer
        x = self.fc2(x) # Output layer

# Evaluate the model
Image Path: dataset/test/crosswalk/1.jpg
Predicted Class: Crosswalk
Confidence: 100.00%
Image Path: dataset/test/stair/1.jpg
Predicted Class: Stair
Confidence: 99.16%
C:\Users\Admin\OneDrive - Assumption University\Desktop\AI concept\Final Exam>
```

Visual Studio Code interface showing a Jupyter Notebook (Arjan_Train.ipynb) and a Python file (Train.py). The Explorer pane on the left shows the project structure, including a folder named "FINAL EXAM" containing "dataset", "model.pth", "Test.py", and "Train.py".

The Jupyter Notebook cell displays the following code:

```
3 import torch.nn.functional as F
4 from PIL import Image
5 import torch
6 import matplotlib.pyplot as plt
7 import os
8 from torchvision import transforms
9
10 # Define the model
11 class SimpleNet(torch.nn.Module):
12     def __init__(self):
13         super(SimpleNet, self).__init__()
14         self.conv1 = torch.nn.Conv2d(3, 16, kernel_size=5)
15         self.conv2 = torch.nn.Conv2d(16, 16, kernel_size=5)
16         self.fc1 = torch.nn.Linear(16 * 5 * 5, 120)
17         self.fc2 = torch.nn.Linear(120, 10)
18
19     def forward(self, x):
20         x = F.relu(F.max_pool2d(self.conv1(x), 2))
21         x = F.relu(F.max_pool2d(self.conv2(x), 2))
22         x = torch.flatten(x, 1)
23         x = F.relu(self.fc1(x))
24         x = self.fc2(x)
25         return F.softmax(x, dim=0)
```

The output of the notebook shows a prediction for a bridge image:

Predicted: Bridge, Confidence: 85.56%



Visual Studio Code interface showing a Jupyter Notebook (Arjan_Train.ipynb) and a Python file (Train.py). The Explorer pane on the left shows the project structure, including a folder named "FINAL EXAM" containing "dataset", "model.pth", "Test.py", and "Train.py".

The Jupyter Notebook cell displays the following code:

```
77 plt.show()
78
79 # Print details
80 print(f"Image Path: {image_path}")
81 print(f"Predicted Class: {predicted_class}")
82 print(f"Confidence: {confidence}")
83
84 # Image 2
85 image_path = dataset.test.crosswalk[0]
86 if not os.path.exists(image_path):
87     raise FileNotFoundError
88
89 image = Image.open(image_path)
90 transform = transforms.Compose([
91     transforms.Resize(256),
92     transforms.CenterCrop(224),
93     transforms.ToTensor(),
94 ])
95 image_tensor = transform(image)
96
97 # Perform prediction
98 with torch.no_grad():
99     output = model(image_tensor.unsqueeze(0))
100     predicted_class = torch.argmax(output, dim=1).item()
101     confidence = torch.nn.functional.softmax(output, dim=1)[0][predicted_class].item()
```

The output of the notebook shows a prediction for a crosswalk image:

Predicted: Crosswalk, Confidence: 100.00%



Visual Studio Code interface showing a Python script execution and a plot window.

EXPLORER

- OPEN EDITORS
 - Test.py
 - Arjan_Train.ipynb
 - Train.py
- FINAL EXAM
 - dataset
 - test
 - Bridge
 - Crosswalk
 - Stair
 - train
 - model.pth
 - Test.py
 - Train.py

Test.py

```
107 plt.axis('off')
108 plt.show()
109
110 # Print det
111 print(f'Ima
112 print(f'Pre
113 print(f'Con
114
115 #image3
116 image_path
117 if not os.p
118     raise F
119
120 image = Im
121 transform =
122 transform
123 transform
124 })
125 image_tenso
126
127 # Perform p
```

Figure 1

Predicted: Stair, Confidence: 99.16%



PROBLEMS **OUTPUT**

Image Path: dataset
Predicted Class: Bridge
Confidence: 85.56%

Image Path: dataset/test/crosswalk/1.jpg
Predicted Class: Crosswalk
Confidence: 100.00%

Ln 116, Col 33 Spaces: 4 UTF-8 CRLF Python 3.11.0 64-bit (3.11.0: pyenv) Go Live Continue

9:30 AM 3/7/2023